ICSPL

Flutter Recruitment Assignment: "Context-Aware Recipe Discovery"

Objective

Build a production-grade Flutter application that demonstrates proficiency in asynchronous programming, local persistence, Location/Time, and automated CI/CD pipelines.

The Challenge

Users struggle to decide what to eat. Build an app that suggests recipes not just by search, but by where they are and what time it is, ensuring the experience remains functional even without an internet connection.

1. Functional Requirements

A. Smart Discovery & Search

- Data Sourcing: Integrate with a public recipe API (e.g., TheMealDB).
- Contextual Logic:
- Time-Based: Suggest "Breakfast" categories in the morning, "Lunch" at midday, and "Dinner" in the evening.
- Location-Based: Use the device's location to prioritize recipes or cuisines relevant to the user's current region/country.
- Search: Implement a search feature with an optimization strategy (e.g., debouncing) to prevent unnecessary API overhead.

B. Offline-First Experience

- Persistence: Users must be able to "Favorite" recipes for offline viewing.
- Caching: Ensure previously viewed recipe data and images are accessible without an active internet connection.
- State Resilience: If the network fails, the app should gracefully transition to showing cached or favorited content rather than an empty screen.

C. Proactive Engagement

- Scheduled Notifications: Trigger a notification at relevant meal times (e.g., at 8:00 AM) suggesting a specific recipe. For example suggest a breakfast recipe at 8:00 AM, Lunch recipe at 2:00 PM etc.
- Permission Handling: Gracefully handle scenarios where the user denies Location or Notification permissions.

2. Technical Constraints

- State Management: Use a robust pattern—Bloc, Riverpod, or MobX. (Avoid vanilla StateSetters for global logic).

UI/UX:

- Perceived Performance: Implement Shimmer/Skeleton loaders for all async operations.
- Micro-interactions: Add an implicit animation for the "Favorite" heart icon and smooth Hero transitions between the list and detail views.
- Error Handling: Implement a global Error UI (e.g., Snackerbars or "No Internet" empty states).

3. DevOps & Delivery

- CI/CD Pipeline: Define a .github/workflows/main.yml that:

1. Runs flutter analyze and flutter test.
2. Builds a Release APK.
3. Automatically uploads the APK to GitHub Releases on every push to the main branch.

Submission Instructions

1. Repository: Provide a private/public GitHub link.
2. Documentation: A README.md explaining your architectural choices and instructions on how to run the CI/CD trigger.
3. Artifact: Ensure the GitHub Releases section contains at least one successfully built APK.